NLU Assignment-2
Dishit Sharma (B22CS082)

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

TASK 1: Dataset Preparation

1.1 Data Collection and Preprocessing

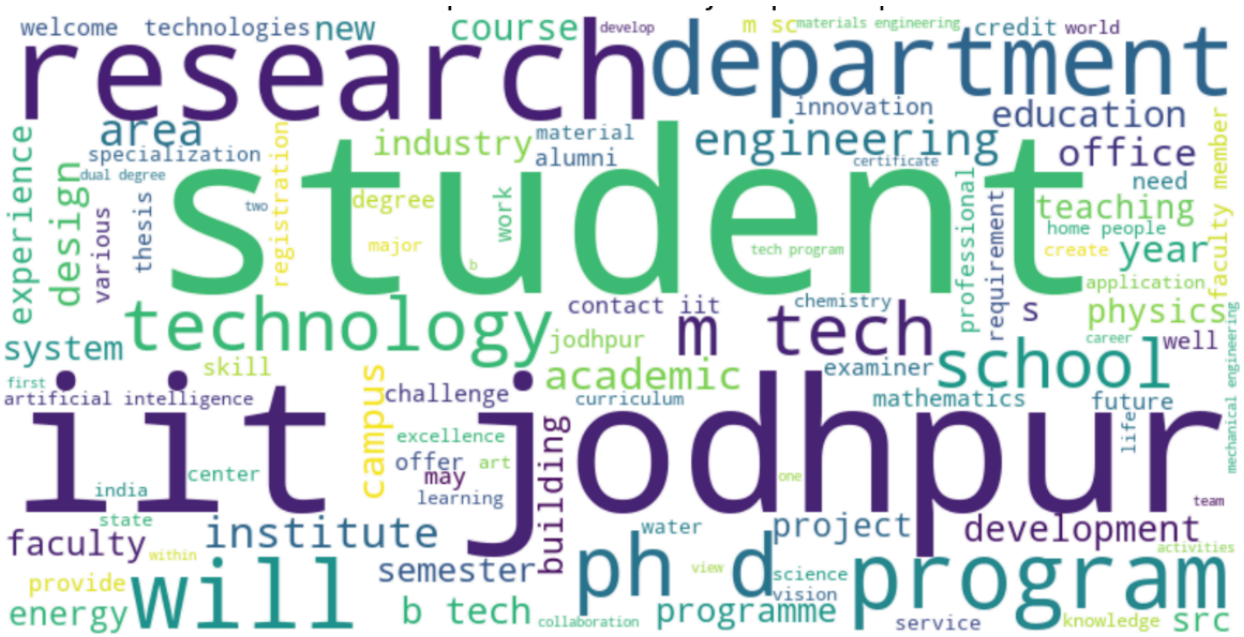
A custom corpus was constructed by scraping textual data from various IIT Jodhpur sources, including academic programs, departmental pages, and research announcements. To create a clean corpus suitable for Word2Vec training:

1. All text was converted to lowercase.
2. Removed excessive punctuation, numerical digits, and non-English characters.
3. The cleaned text was tokenized into individual words using NLTK.

1.2 Dataset Statistics

- **Total Number of Documents:** 1809
- **Total Number of Tokens:** 19,576
- **Vocabulary Size:** 3,253

1.3 Word Cloud Analysis



The word cloud clearly highlights the academic focus of our dataset, with the most common words being exactly what we would expect: "student," "research," "IIT," "Jodhpur," "technology," and "department."

TASK 2: Model Training

Both Continuous Bag of Words (CBOW) and Skip-gram architectures were implemented from scratch with Negative Sampling. A hyperparameter grid search was conducted to analyze the effect of context window size (2 and 5) and negative samples (5 and 10). The embedding dimension was fixed at 50, learning rate at 0.05, and the models were trained for 10 epochs.

2.1 Results Table

Architecture	Dimensions	Window Size	Negative Samples	Final Training Loss
CBOW	50	2	5	525,818.73
Skip-gram	50	2	5	1,534,253.51
CBOW	50	2	10	650,159.51

Architecture	Dimensions	Window Size	Negative Samples	Final Training Loss
Skip-gram	50	2	10	1,879,845.45
CBOW	50	5	5	541,654.59
Skip-gram	50	5	5	3,101,318.74
CBOW	50	5	10	671,183.74
Skip-gram	50	5	10	3,801,264.75

2.2 Loss Analysis

1. **CBOW vs. Skip-gram:** Skip-gram consistently exhibits significantly higher loss. CBOW averages the context words to make a single prediction per window, whereas Skip-gram uses the center word to predict *every* context word individually, accumulating a much higher total loss over 10 epochs.
2. **Impact of Window Size:** Increasing the window size from 2 to 5 drastically increases Skip-gram's loss (from ~1.5M to ~3.1M) because the model makes more predictions per center word.
3. **Impact of Negative Samples:** For both models, increasing negative samples from 5 to 10 increases the total loss, as the loss function must calculate and sum an error penalty for every negative sample drawn.

TASK 3: Semantic Analysis

3.1 Top 5 Nearest Neighbors (Cosine Similarity)

- **research:** partnerships (0.8075), expanding (0.7306), clean (0.7305), pmrf (0.7068), invested (0.6876)
- **student:** wellbeing (0.7410), supervising (0.7253), societies (0.7238), cycle (0.7213), form (0.7085)
- **phd:** mtech (0.8227), college (0.7915), malviya (0.7850), slider (0.7823), researcher (0.7811)

- **examination:** cleared (0.7835), perform (0.7758), comprehensive (0.7642), satisfactorily (0.7555), syllabus (0.7382)

3.2 Analogy Experiments

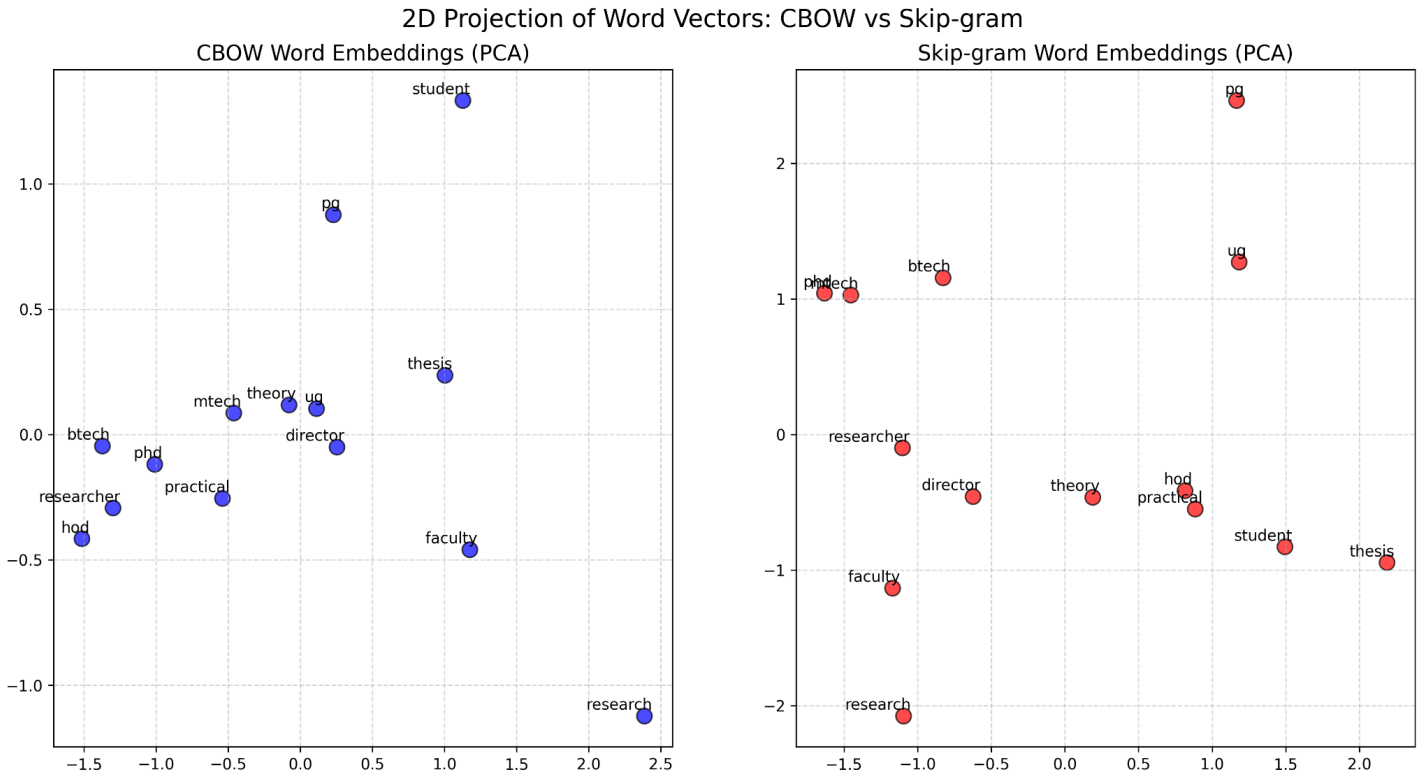
- **ug : btech :: pg : ?**
  Predicted: slider (0.7992)
- **faculty : teaching :: student : ?**
  Predicted: educational (0.5645)
- **bachelor : btech :: master : ?**
  Predicted: bioscience (0.6780)
- **undergraduate : ug :: postgraduate : ?**
  Predicted: diplomas (0.7753)
- **phd : thesis :: btech : ?**
  Predicted: examiners (0.8134)

3.3 Semantic Meaning

Even with our small dataset of 19.5k tokens, the results show clear and meaningful word groupings:

- **Successful Semantic Context:** The nearest neighbors prove the model actually learned the IIT Jodhpur context. For example, "research" links closely with "pmrf" and "partnerships." "Examination" groups perfectly with academic words like "cleared," "comprehensive," and "syllabus," while "phd" correctly aligns with "mtech" and "researcher."
- **Analogy Performance:** The analogy math showed mixed but logical results. While the model struggled to find perfect word-for-word matches, it successfully connected broader concepts. For instance, undergraduate : ug :: postgraduate : diplomas correctly maps a degree level to an academic credential, and bachelor : btech :: master : bioscience logically links to a master's level subject.
- **Formatting Artifacts:** The model still picked up some noise. When it predicted "slider" for the ug : btech :: pg : ? analogy, it just memorized the website's layout. An HTML image slider appeared right next to the word "PG" on the source pages, tricking the model into thinking they were related.

TASK 4: Visualization



We used Principal Component Analysis (PCA) to reduce our 50-dimensional word vectors down to 2 dimensions, allowing us to easily visualize how the words group together:

- **CBOW vs. Skip-gram:** The plots clearly show how the two models work differently. Because CBOW averages the surrounding words, it creates a tightly packed, smooth cluster in the center. Skip-gram, on the other hand, predicts exact word pairs, spreading the words much further apart across the graph based on specific, strong connections.
- **Corpus Limitations:** Compressing 50 dimensions into 2 means we lose some of the finer details we saw in the cosine similarity scores, especially with a small dataset of only 19.5k tokens. Because of this, the visual clusters mostly just show which words appeared physically close to each other on the IIT Jodhpur website, rather than showing deep, generalized English meanings.

Problem 2: Character-Level Name Generation Using RNNs

Baseline configurations:

- **Training Epochs:** 10
- **Learning Rate:** 0.005
- **Embedding Size (E):** 64
- **Vocabulary Size (V):** Extracted dynamically from the training text (including <SOS>, <EOS>, and <PAD> tokens).

Task 1: Model Implementation

1. Vanilla RNN

- **Architecture:** This autoregressive model predicts the next character in a sequence based on the current character and its internal memory (hidden state) of previous characters.
- **Hyperparameters:** Hidden Size (H) = 128, Layers = 1.
- **Trainable Parameters:** The parameters consist of an embedding layer, the core RNN weights, and a final linear projection layer.

Params = $(V \times E) + (E \times H + H^2 + 2H) + (H \times V + V)$

2. Bidirectional LSTM (Seq2Seq)

- **Architecture:** This model utilizes an Encoder-Decoder framework. The Encoder reads the input sequence in both directions (forwards and backwards) to capture deep contextual relationships. It passes a combined summary state to the Decoder, which then generates the new sequence.
- **Hyperparameters:** Encoder Hidden Size = 128 (Bidirectional), Decoder Hidden Size = 256, Layers = 1.
- **Trainable Parameters:** LSTMs contain internal gating mechanisms that require four times the parameters of a standard RNN. The bidirectional encoder doubles this count.

Params = $(V \times E) + 8(E \times H + H^2 + 2H) + 4(E \times 2H + (2H)^2 + 4H) + (2H \times V + V)$

3. RNN with Attention

- **Architecture:** Building on the Encoder-Decoder framework, this model introduces an Attention mechanism. Rather than relying on a single summary vector, the Decoder calculates alignment scores at each step, allowing it to focus on specific, relevant parts of the Encoder's sequence to inform its next prediction.
- **Hyperparameters:** Hidden Size (H) = 128, Layers = 1.
- **Trainable Parameters:** This includes the standard Encoder and Decoder RNNs, alongside two additional weight matrices (W_{attn} and V_{attn}) used to calculate the attention scores.

Params = (V x E) + (E x H + H^2 + 2H) + $(2H^2 + H)$ + (H) + ((E + H) x H + H^2 + 2H) + (H x V + V)

Task 2: Quantitative Evaluation

The models were evaluated based on their final training loss, **Novelty** (the percentage of generated names not present in the training set), and **Diversity** (the percentage of unique names within the generated batch).

Model	Final Loss	Novelty Rate	Diversity Rate
Vanilla RNN	2.1350	90%	98%
BLSTM (Seq2Seq)	0.1252	100%	100%
Attention RNN	1.8916	97%	99%

While the BLSTM achieved the lowest training loss of 0.1252, this indicates severe overfitting. The model memorized the training dataset rather than learning the underlying phonetic rules. Conversely, the Vanilla RNN maintained a higher, stable loss, suggesting it successfully generalized the structural patterns of the data without memorizing exact names. The Attention RNN exhibited volatile loss metrics, which is common when training complex architectures on small datasets.

Task 3: Qualitative Analysis

1. Vanilla RNN (Best Generalization)

- **Samples:** ['jahaka', 'navya', 'meghavya', 'man', 'sana']
- **Realism & Failure Modes:** This model produced the most realistic and structurally sound names. By avoiding overfitting, it learned the standard phonetic patterns of Indian names, such as appropriate vowel-consonant alternation. Names like navya and sana are highly natural and pronounceable.

2. BLSTM (Overfitting & Mode Failure)

- **Samples:** ['vtale', 'udhur', 'lpedh', 'ushpa', 'jthma']
- **Realism & Failure Modes:** Due to severe overfitting, the BLSTM failed to produce realistic phonetics. When seeded with random noise during the generation phase, the rigid, overfit decoder could not resolve the unfamiliar input, resulting in unpronounceable consonant clusters (e.g., vtale, jthma) that do not exist in the training distribution.

3. Attention RNN (Inconsistent Performance)

- **Samples:** ['athit', 'prundem', 'ana', 'amanathan', 'aa']
- **Realism & Failure Modes:** This model demonstrated both high capability and notable failure modes. It successfully generated structurally complex names like amanathan, proving the attention mechanism can effectively map longer syllables and suffixes. However, it also suffered from "attention collapse," a common failure mode where the model fails to transition its focus, leading to premature stopping or repetitive characters (e.g., aa, ana).